

Introduction to FastAPI

1. What is FastAPI?

FastAPI is a modern, high-performance web framework for building APIs with Python. It is built on top of Starlette for the web parts and Pydantic for data validation, and it takes full advantage of Python's type hints. FastAPI was created by Sebastian Ramirez and has become one of the most popular frameworks for building RESTful APIs in Python.

Unlike many older frameworks, FastAPI was designed from the ground up to support asynchronous programming, automatic data validation, and automatic interactive documentation, making it a strong choice for building production-grade APIs quickly.

2. Why FastAPI is Used

- **High performance:** Comparable to NodeJS and Go, thanks to Starlette and async support.
- **Fast to code:** Type hints and automatic validation reduce development time significantly.
- **Fewer bugs:** Automatic data validation catches many errors before they reach production.
- **Great developer experience:** Editor autocompletion and type checking work out of the box.
- **Standards-based:** Fully compatible with OpenAPI and JSON Schema specifications.

3. Main Features

- Automatic request validation using Python type hints and Pydantic models.
- Automatic generation of interactive API documentation (Swagger UI and ReDoc).
- Native support for asynchronous request handling with async/await.
- Dependency injection system for managing shared logic like authentication.
- Built-in support for security schemes such as OAuth2 and API keys.

4. Installation

FastAPI can be installed using pip. You will also need an ASGI server such as uvicorn to run the application.

```
pip install fastapi
pip install "uvicorn[standard]"
```

5. Creating a Basic FastAPI Application

A minimal FastAPI application can be created in just a few lines of code. The example below defines a single endpoint that returns a JSON response.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}
```

The application can then be started with uvicorn:

```
uvicorn main:app --reload
```

6. GET and POST Endpoints

FastAPI provides decorators for each HTTP method. A GET endpoint is typically used to retrieve data, while a POST endpoint is used to create new data.

```
@app.get("/items")
def list_items():
    return ["item1", "item2"]

@app.post("/items")
def create_item(name: str):
    return {"created": name}
```

7. Path and Query Parameters

Path parameters are declared as part of the URL, while query parameters are passed after a question mark in the URL. FastAPI automatically parses and validates both, based on the type hints used in the function signature.

```
@app.get("/users/{user_id}")
def get_user(user_id: int, active: bool = True):
    return {"user_id": user_id, "active": active}
```

In this example, `user_id` is a required path parameter, while `active` is an optional query parameter with a default value.

8. Request Body and Pydantic Models

To accept structured data in the body of a request, FastAPI uses Pydantic models. A Pydantic model defines the expected fields and their types, and FastAPI automatically validates incoming JSON data against it.

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True

@app.post("/items/")
def create_item(item: Item):
    return item
```

9. Automatic API Documentation

One of FastAPI's standout features is automatic, interactive API documentation. Once an application is running, developers can visit `/docs` for a Swagger UI interface, or `/redoc` for an alternative ReDoc-based interface, both generated automatically from the code.

10. Async and Await

FastAPI natively supports asynchronous endpoint functions using Python's `async/await` syntax. Async endpoints are especially useful when handling I/O-bound operations such as database queries or external API calls, since they allow the server to handle other requests while waiting for those operations to complete.

```
import asyncio

@app.get("/async-data")
async def get_data():
    await asyncio.sleep(1)
    return {"status": "done"}
```

11. Common Use Cases

- Building backend REST APIs for web and mobile applications.
- Serving machine learning models through prediction endpoints.
- Creating microservices that communicate over HTTP.
- Building internal tools and dashboards with fast iteration cycles.
- Powering real-time applications using WebSockets.

12. Conclusion

FastAPI combines speed, simplicity, and modern Python features to make API development faster and less error-prone. Its automatic validation, dependency injection system, and built-in documentation reduce boilerplate code and help developers ship reliable APIs more quickly. For anyone building Python-based web services, FastAPI is a strong and widely adopted choice.